AI Agent-Based Autonomous Ticket Booking Application

Team: The Xplorers | B.E., Computer Science and Engineering (First Year) | Team Size: 4

1. Project Title

AI Agent-Based Autonomous Ticket Booking Application — a conversational system in which a user describes a travel need in plain language (e.g., “Book a flight from Delhi to Mumbai tomorrow for two people”) and an AI agent searches, confirms, and books the ticket.

2. Aim of the Project

To build a working website (and later a mobile app from the same codebase) in which an AI chat agent autonomously completes the full ticket-booking workflow: understanding the request, searching live options through a booking API, taking user confirmation, issuing the ticket, and supporting booking history and cancellation.

3. Problem Statement

Booking travel tickets online still takes manual effort — searching across sites, comparing price and timing, selecting seats, and filling long passenger and payment forms. Real booking (seat locking, payment, ticket issuance) is far harder than only showing prices, and transport APIs differ across trains, buses, and flights, which makes one end-to-end autonomous flow difficult for a student team to build.

4. Proposed Solution

A chat-first web application. The user types the trip details; the agent extracts origin, destination, date, passenger count, and class, then calls backend tools in order: (1) search trips through a flight booking API, (2) show the best options with fare and timing, (3) ask for explicit confirmation with passenger details, (4) create the order and store the booking reference (PNR), and (5) display the e-ticket with history and cancellation support. The API key stays only on the server. Initial scope is one transport (flights) to prove the full loop; the same design extends to other transports later.

5. Concepts Used

- AI agents and tool (function) calling — the model chooses which backend function to run with structured inputs.
- Slot filling and confirmation — collecting missing fields (date, passengers, names) before any payment step.
- Client–server architecture and REST API integration (search, order, cancel).
- Authentication, database storage, and row-level access control for per-user history.
- Responsive web build reused as an installable mobile app (no rewrite).

6. Technologies Used

- Next.js (App Router) with API routes for frontend and backend; Tailwind CSS with shadcn/ui components.
- Vercel AI SDK for streaming chat and tool definitions.
- Supabase (PostgreSQL, authentication, row-level security) for data and login.
- Duffel Flights API — free test mode for development; the traveller pays the actual fare in live use.
- Hosting on Vercel; mobile path via Progressive Web App plus Capacitor wrapper from the same build.

7. Programming Languages

TypeScript / JavaScript (application and AI tool logic), SQL (PostgreSQL tables for searches and bookings), HTML and CSS via Tailwind CSS for layout and styling.